

자구&알고 스터디

재귀 (Recursive)

분할 정복 (Divide And Conquer)

재귀

- 자기 자신을 호출하는 함수
- 예시:

1. 학생이 책을 찾음 → 교실에 없어서 옆 반에 부탁
2. 옆 반에 없어서 또 옆 반에 부탁
3. 또 없어서 도서관에 부탁
4. 도서관에 책이 있어서 찾음
→ 3. 옆 반으로 책 전달
→ 2. 다시 첫 번째 옆 반으로 전달
→ 1. 학생이 책을 받음



```
string findBook(Room current) {  
    // base case: 책이 있으면 바로 반환  
    if (current.hasBook())  
        return "책 발견!";  
    // 더 이상 옆 방이 없으면 실패  
    if (current.nextRoom == NULL)  
        return "책 없음";  
    // 재귀적으로 옆 방에 부탁  
    return findBook(current.nextRoom);  
}
```

재귀

- 접근하려는 문제가 똑같은 문제로 나눌 수 있을 때 유용함
 - 점화식 계산, 완전 탐색, 반복문으로 하기 힘든 작업 등등에서 사용
 - ex. 1~5 까지 더하시오
 - 이 문제는 "1~4 까지 더한 뒤 5 를 더하시오"로 문제를 나눌 수 있음
 - $f(n) = f(n - 1) + n$
 - ex. 피보나치 수열 구하기
 - $f(n) = f(n - 1) + f(n - 2)$
 - ex. 팩토리얼 구하기
 - $f(n) = f(n - 1) \times n$
- 접근하려는 문제를 단계별로 접근할 때, 이전 단계를 똑같은 문제로 접근하려 할 때

재귀

- 베이스 케이스: 재귀 호출을 멈추는 조건
 - 초기값을 직접 알 수 있는 경우
 - 손으로 쉽게 풀 수 있는 수준까지 줄어든 경우
 - 자명하게 답을 알 수 있는 단계
- 이전 단계까지의 과정이 잘 수행되었다고 믿고 다음 단계를 수행함을 전제함

재귀

```
long long getSum(int n) {  
    if (n == 1) // base case  
        return 1;  
    else {  
        long long result = n + getSum(n - 1);  
        return result;  
    }  
}
```

```
int main() {  
    int x;  
    cin >> x;  
  
    cout << getSum(x);  
}
```

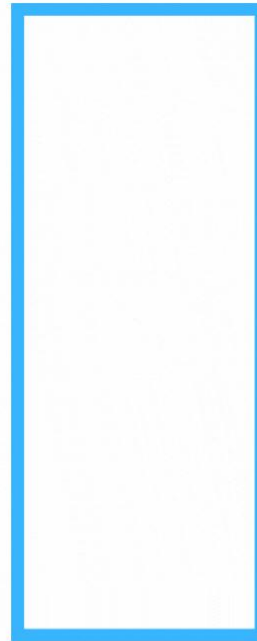
재귀

- 예시: 1부터 5까지 더해보기

Recursion

```
function addAll(n){  
  if(n === 1){return 1;}  
  else {return n + addAll(n - 1)}  
}
```

Call Stack



재귀

- 단, 재귀는 웬만하면 반복문으로 구현 가능.
- 재귀는 코드를 더 깔끔하게, 또는 논리적으로 이해하기 쉽게 적을 수 있는 경우가 있다 장점 있음.
- 단점으로 반복문보다 메모리를 훨씬 더 많이 잡아먹는다는 단점이 있다.
- 재귀는 일종의 도구 중 하나로 생각하는게 오히려 더 편함.

재귀 - 예시문제

- 팩토리얼: <https://www.acmicpc.net/problem/27433>
 - 예시 답안 (C++) : <http://boj.kr/274fa52aecb6428e8fdc743a57c05550>
 - 예시 답안 (Java) : <http://boj.kr/17d872eff05a4296bf834aab54a9097d>

분할 정복

- 분할 - 큰 문제를 여러 개의 작은 문제로 쪼갠 후
- 정복 - 작은 문제들의 답을 이용해 큰 문제의 답을 구하는 방법
- 문제의 크기가 충분히 작으면 직접 해결한다.
- 그렇지 않으면 작은 문제 여러 개로 분할해 해결한 후 그 결과를 이용해 문제를 해결한다.

분할 정복

- 예시: $a^b \bmod c$ 빠르게 계산하기
 - 만약 b 가 1000만 이하라면 어떻게 구현하던 선형 시간 내에 빠르게 해결 가능하다.
 - 하지만 b 가 1억 정도 되면 다항 시간 내에 해결해야 한다.
 - ex. $1354757413857^{4698764156374654} \bmod 368561265741$
 - 이 때, 우리는 분할 정복을 이용할 수 있다.
 - 어디까지 분할해야 충분히 작은 문제일까?
 - 분할해서 가져온 결과를 어떻게 사용해야 할까?

분할 정복

- 예시: $a^b \bmod c$ 빠르게 계산하기
- $\{(a \bmod e) * (a \bmod e)\} \bmod e = (a * b) \bmod e$
 - $b = 0$ 이면 충분히 작기에 직접 해결한다. (base case : $a^0 = 1$)
 - $b \geq 1$ 이면 더 작은 문제로 나눠 해결한다.
 - B가 짝수면 $a^{b/2} \times a^{b/2} \pmod c$
 - B가 홀수면 $a^{(b-1)/2} \times a^{(b-1)/2} \times a \pmod c$
- 시간 복잡도
 - b의 최상위 비트까지 내려가니 $O(\log b)$ ($\log_2 b$ 가 맞지만 대부분 $\log b$ 으로 표현)

분할 정복

- 예시: $a^b \bmod c$ 빠르게 계산하기
 - ex. 3^5 계산하기
 - $f(3, 5) \Rightarrow f(3, 2)^2 \times 3$ (b가 홀수)
 - $f(3, 2) \Rightarrow f(3, 1)^2$ (b가 짝수)
 - $f(3, 1) \Rightarrow f(3, 0)^2 \times 3$ (b가 홀수)
 - $f(3, 0) \Rightarrow 1$ (b가 base case)
 - $f(3, 5) = (1 \times 3)^{2^2} \times 3$

분할 정복 - 예시문제

- $a^b \bmod c$: <https://www.acmicpc.net/problem/1629>
 - 예시 답안 (C++) : <http://boj.kr/6ec96420913547fca1615d2215add62b>
 - 예시 답안 (Java) : <http://boj.kr/f2fced4ddffb45ed9593a3155e8e459e>

복습 문제

- 재귀 : <https://www.acmicpc.net/step/19>
- 분할 정복 : <https://www.acmicpc.net/step/20>